

WEEK 7 Assignment

Step 1: Import Required Libraries

```
!pip install -q delta-spark
```

53.9/53.9 kB 1.9 MB/s eta 0:00:00

```
from pyspark.sql import SparkSession
from delta import configure_spark_with_delta_pip

builder = SparkSession.builder \
    .appName("Week7_DeltaLake") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")

spark = configure_spark_with_delta_pip(builder).getOrCreate()

print("Spark Version:", spark.version)
```

Spark Version: 4.0.3

Step 2: Load the Dataset

```
df = spark.read.csv(
    "/content/superstore.csv",
    header=True,
    inferSchema=True,
    encoding="ISO-8859-1"
)
```

```
df.show(5)
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Hendersonville
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Hendersonville
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale

only showing top 5 rows

```
df.printSchema()
```

```
root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: string (nullable = true)
|-- Ship Date: string (nullable = true)
|-- Ship Mode: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Segment: string (nullable = true)
|-- Country: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Postal Code: integer (nullable = true)
|-- Region: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sub-Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Sales: string (nullable = true)
|-- Quantity: string (nullable = true)
|-- Discount: string (nullable = true)
|-- Profit: double (nullable = true)
```

Step 4. Save Dataset as Delta Table

```
delta_path = "/content/delta/superstore"
```

```
# Rename columns to remove invalid characters (spaces) for Delta Lake
import re
for col_name in df.columns:
```

```

if ' ' in col_name:
    new_col_name = re.sub(r'^a-zA-Z0-9_', '_', col_name)
    df = df.withColumnRenamed(col_name, new_col_name)

df.write \
    .format("delta") \
    .mode("overwrite") \
    .save(delta_path)

```

```

delta_df = spark.read.format("delta").load(delta_path)

delta_df.show(5)

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Row_ID|   Order_ID|Order_Date|Ship_Date|   Ship_Mode|Customer_ID|  Customer_Name|Segment|   Country|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|   1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|   Claire Gute|Consumer|United States|Hender
|   2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|   Claire Gute|Consumer|United States|Hender
|   3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|  DV-13045|Darrin Van Huff|Corporate|United States|Los Ange
|   4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|  SO-20335|  Sean O'Donnell|Consumer|United States|Fort Lauderc
|   5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|  SO-20335|  Sean O'Donnell|Consumer|United States|Fort Lauderc
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Step 5. Basic Cleaning

```
clean_df = delta_df.dropna()
```

For Removing Duplicate Records

```
clean_df = clean_df.dropDuplicates()
```

```

clean_df.write \
    .format("delta") \
    .mode("overwrite") \
    .save(delta_path)

```

Step 6. Creating Incremental Dataset

```
incremental_df = clean_df.limit(5)
```

```

from pyspark.sql.functions import lit

incremental_df = incremental_df.withColumn(
    "Sales",
    lit(9999.99)
)

```

```

new_record = spark.createDataFrame([
    (
        10001,
        "CA-2027-111111",
        "1/1/2027",
        "1/5/2027",
        "Second Class",
        "CG-99999",
        "Anushka",
        "Consumer",
        "United States",
        "Nashik",
        "Maharashtra",
        422001,
        "West",
        "FUR-NEW",
        "Furniture",
        "Chairs",
        "Office Chair",
        5500.0,
        2,
        0.0,
        900.0
    )
], clean_df.columns)

incremental_df = incremental_df.union(new_record)

```

```
incremental_df.show()
```

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	City
37	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	Gene Hale	Corporate	United States	Richardsor
162	CA-2015-119697	12/28/2015	12/31/2015	Second Class	EM-13960	Eric Murdock	Consumer	United States	Philadelphia
169	CA-2014-139892	9/8/2014	9/12/2014	Standard Class	BM-11140	Becky Martin	Consumer	United States	San Antonio
188	CA-2016-157000	7/16/2016	7/22/2016	Standard Class	AM-10360	Alice McCarthy	Corporate	United States	Grand Prairie
211	CA-2017-135860	12/1/2017	12/7/2017	Standard Class	JH-15985	Joseph Holt	Consumer	United States	Saginaw
10001	CA-2027-111111	1/1/2027	1/5/2027	Second Class	CG-99999	Anushka	Consumer	United States	Nashik

Step 7. MERGE Operation

```
from delta.tables import DeltaTable
```

```
delta_table = DeltaTable.forPath(spark, delta_path)
```

```
(
    delta_table.alias("target")
    .merge(
        incremental_df.alias("source"),
        "target.Order_ID = source.Order_ID"
    )
    .whenMatchedUpdate(set={
        "Sales": "source.Sales",
        "Profit": "source.Profit"
    })
    .whenNotMatchedInsertAll()
    .execute()
)
```

```
DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]
```

Step 8. Result

```
final_df = spark.read.format("delta").load(delta_path)
```

```
print("Total Rows :", final_df.count())
```

```
Total Rows : 9995
```

```
duplicates = final_df.count() - final_df.dropDuplicates().count()
```

```
print("Duplicate Records :", duplicates)
```

```
Duplicate Records : 0
```

```
from pyspark.sql.functions import col, count, when
```

```
final_df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in final_df.columns
]).show()
```

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	City	State	Postal_Code	Region	Proc
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Step 9. Final Dataset

```
final_df.show(20, truncate=False)
```

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	City
2718	CA-2014-100006	9/7/2014	9/13/2014	Standard Class	DK-13375	Dennis Kane	Consumer	United States	New York
6288	CA-2014-100090	7/8/2014	7/12/2014	Standard Class	EB-13705	Ed Braxton	Corporate	United States	San Francisco
6289	CA-2014-100090	7/8/2014	7/12/2014	Standard Class	EB-13705	Ed Braxton	Corporate	United States	San Francisco
9515	CA-2014-100293	3/14/2014	3/18/2014	Standard Class	NF-18475	Neil Französisch	Home Office	United States	Jacksonville
3084	CA-2014-100328	1/28/2014	2/3/2014	Standard Class	JC-15340	Jasper Cacioppo	Consumer	United States	New York

9441	CA-2014-100391	5/25/2014	5/29/2014	Standard Class	BW-11065	Barry Weirich	Consumer	United States	New York
9021	CA-2014-100706	12/16/2014	12/18/2014	Second Class	LE-16810	Laurel Elliston	Consumer	United States	Springfie
9020	CA-2014-100706	12/16/2014	12/18/2014	Second Class	LE-16810	Laurel Elliston	Consumer	United States	Springfie
6317	CA-2014-100762	11/24/2014	11/29/2014	Standard Class	NG-18355	Nat Gilpin	Corporate	United States	Jackson
6318	CA-2014-100762	11/24/2014	11/29/2014	Standard Class	NG-18355	Nat Gilpin	Corporate	United States	Jackson
6316	CA-2014-100762	11/24/2014	11/29/2014	Standard Class	NG-18355	Nat Gilpin	Corporate	United States	Jackson
6315	CA-2014-100762	11/24/2014	11/29/2014	Standard Class	NG-18355	Nat Gilpin	Corporate	United States	Jackson
7573	CA-2014-100916	10/21/2014	10/26/2014	Standard Class	FH-14275	Frank Hawley	Corporate	United States	Newport M
7572	CA-2014-100916	10/21/2014	10/26/2014	Standard Class	FH-14275	Frank Hawley	Corporate	United States	Newport M
7574	CA-2014-100916	10/21/2014	10/26/2014	Standard Class	FH-14275	Frank Hawley	Corporate	United States	Newport M
7868	CA-2014-100972	11/19/2014	11/24/2014	Second Class	DB-13360	Dennis Bolton	Home Office	United States	Salt Lake
8091	CA-2014-101392	12/7/2014	12/13/2014	Standard Class	AS-10630	Ann Steele	Home Office	United States	Seattle
8690	CA-2014-101427	12/26/2014	12/30/2014	Standard Class	AY-10555	Andy Yotov	Corporate	United States	Philadelp
428	CA-2014-101476	9/12/2014	9/13/2014	First Class	SD-20485	Shirley Daniels	Home Office	United States	New Roche
5480	CA-2014-101770	3/31/2014	4/4/2014	Standard Class	KB-16240	Karen Bern	Corporate	United States	Miami

only showing top 20 rows

```
print("Original Rows :", df.count())
print("Rows After Cleaning :", clean_df.count())
print("Rows After Merge :", final_df.count())
```

```
Original Rows : 9994
Rows After Cleaning : 9995
Rows After Merge : 9995
```

Loaded the Superstore dataset into a Delta table. Cleaned the data by removing null values and duplicate records. Created an incremental dataset containing updated and new records. Used the Delta Lake MERGE operation to update existing records and insert new records. Validated the final dataset by checking the total row count, duplicate records, and null values. Displayed the final Delta table along with a summary of the processing steps.

Outcome: The assignment demonstrated how Delta Lake supports efficient incremental data processing by combining update and insert operations into a single MERGE statement while maintaining data consistency.

Start coding or [generate](#) with AI.